

RAG-Based Customer Support Assistant

Low-Level Design Document

Submitted by: Bishwadeo

1. Introduction

This document describes the internal implementation details of the **RAG-based Customer Support Assistant**. It provides a detailed breakdown of system modules, data structures, workflow, and logic used in the system.

2. Module-Level Design

2.1 Document Processing Module

This module loads the PDF file using a document loader and extracts textual content.

2.2 Chunking Module

Splits the extracted text into smaller chunks using a recursive text splitter.

- Chunk Size: 500
 - Overlap: 50
-

2.3 Embedding Module

Converts text chunks into vector embeddings using a HuggingFace embedding model.

2.4 Vector Storage Module

Stores embeddings in **ChromaDB**, enabling efficient similarity search.

2.5 Retrieval Module

Retrieves top relevant chunks based on semantic similarity using:

- Vector search
 - Top-k retrieval (k = 3)
-

2.6 Query Processing Module

Combines user query with retrieved document chunks for answer generation.

2.7 Graph Execution Module

Implements workflow using LangGraph:

- Maintains state
 - Executes nodes
 - Handles transitions
-

2.8 HITL Module

Handles Human-in-the-Loop escalation when system confidence is low.

3. Data Structures

3.1 State Object

query: string
docs: list
answer: string
confidence: float
escalate: boolean
final_answer: string

3.2 Chunk Structure

text: string
metadata: dictionary

3.3 Query-Response Schema

Input:

```
{  
  "query": "What is refund policy?"  
}
```

Output:

```
{  
  "answer": "...",  
  "confidence": 0.9  
}
```

4. Workflow Design

Nodes

- Retrieve Node
 - Generate Node
 - Decision Node
 - HITL Node
 - Output Node
-

Flow

Input → Retrieve → Generate → Decision → Output / HITL

State Flow

- Query → Docs → Answer → Confidence → Decision → Output
-

5. Conditional Routing Logic

- If **confidence** ≥ 0.5 → return answer
 - If **confidence** < 0.5 → escalate to human
-

6. HITL Design

Trigger Conditions

- No relevant documents
 - Low confidence score
-

Process

1. System detects low confidence
 2. Query is escalated
 3. Human provides response
 4. Response returned to user
-

7. API Design

Input Format

```
{  
  "query": "What is refund policy?"  
}
```

Output Format

```
{  
  "answer": "...",  
  "confidence": 0.9  
}
```

8. Error Handling

- No relevant chunks → escalation
 - Missing PDF → system error
 - Retrieval failure → fallback
 - Unexpected errors → safe response
-

9. Conclusion

This Low-Level Design ensures a modular, scalable, and efficient implementation of the RAG-based system using embeddings, vector search, and workflow-based execution.